HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

INTERNSHIP REPORT

Company: Amazon Web Services Viet Nam Company Limited

Position: Workforce Bootcamp - First Cloud AI Journey

Student Information:

Full Name:	Thai Duc Khang
Class:	CC23KHM
NumID:	2352500
Major:	Computer Science
Email:	khang.thaiduc@hcmut.edu.vn
Phone Number:	0917068787
Duration:	6/6/2026 – 15/08/2026

Ho Chi Minh City, July 31, 2026

Contents

1 Worklog

This worklog records my individual contribution to the **CloudLibrary – AWS Library Borrowing System** from **15 June 2026 to 30 July 2026**. Within the three-member team, Member A was responsible for frontend development, Member B was responsible for backend development, while I was responsible for the remaining work, including system integration, Docker, GitHub workflow management, AWS infrastructure, CI/CD, monitoring, deployment troubleshooting, testing, and technical documentation.

1.1 Week 1 Worklog

Period: 15/06/2026 – 19/06/2026

Week 1 Objectives

- Understand the CloudLibrary project requirements and define the responsibilities of each team member.
- Study the AWS services required to deploy a containerized full-stack application.
- Prepare the source-code management strategy and the initial AWS deployment architecture.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Reviewed the internship requirements, project plan, and expected report structure. Discussed the project scope and divided the team into frontend, backend, and Cloud/DevOps responsibilities.	15/06/2026	15/06/2026	First Cloud AI Journey
3	Studied the roles of IAM, Amazon ECR, Elastic Beanstalk, Amazon S3, CloudWatch, EC2, and AWS CLI in a containerized deployment workflow.	16/06/2026	16/06/2026	AWS Overview
4	Reviewed the existing React frontend and Flask backend source code. Identified the required API path, environment variables, database connection, health-check endpoint, and integration points.	17/06/2026	17/06/2026	Docker Compose documentation
5	Prepared the Git repository structure and branch strategy, including `develop`, `develop_2.0`, and feature branches. Defined commit, merge, and integration rules for the team.	18/06/2026	18/06/2026	GitHub Git documentation
6	Drafted the initial deployment architecture and resource naming convention. Selected `ap-southeast-2` as the AWS region and planned the CI/CD flow from GitHub to AWS.	19/06/2026	19/06/2026	AWS Regions and endpoints

Week 1 Achievements

- Understood the overall CloudLibrary architecture and project scope.
- Defined a clear division of responsibilities for the three-member team.
- Identified the AWS services required for deployment and monitoring.
- Established the Git branch strategy and repository organization.
- Produced the first version of the AWS deployment architecture.

1.2 Week 2 Worklog

Period: 22/06/2026 – 26/06/2026

Week 2 Objectives

- Integrate the frontend, backend, and PostgreSQL services in a local environment.
- Containerize the application with Docker and Docker Compose.
- Verify that the complete application can run locally before AWS deployment.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Created the local integration plan and prepared `.env.example`. Defined environment variables for PostgreSQL, Flask, JWT configuration, application logging, and API routing.	22/06/2026	22/06/2026	Docker environment variables
3	Worked with the backend member to prepare the backend container, Gunicorn startup command, dependency installation, and the `/health` endpoint used by Docker and AWS health checks.	23/06/2026	23/06/2026	Gunicorn documentation
4	Worked with the frontend member to prepare the React production build, Nginx configuration, static asset delivery, and reverse proxy from `/api` to the backend service.	24/06/2026	24/06/2026	Nginx proxy module
5	Created and refined `docker-compose.yml` with three services: `db`, `backend`, and `frontend`. Added service dependencies, health checks, network configuration, and a database volume.	25/06/2026	25/06/2026	Compose file reference
6	Ran the complete application locally, tested authentication and API communication, corrected CORS and API-path issues, and documented the local startup and shutdown commands.	26/06/2026	26/06/2026	Docker Compose up

Week 2 Achievements

- Integrated the frontend, backend, and PostgreSQL services locally.
- Created a working Docker Compose environment for the complete system.
- Configured Nginx as the frontend web server and API reverse proxy.
- Added health checks and controlled the service startup order.
- Verified the main application flow before beginning AWS deployment.

1.3 Week 3 Worklog

Period: 29/06/2026 – 03/07/2026

Week 3 Objectives

- Build a continuous integration workflow for the project.
- Automatically validate backend tests and the frontend production build.
- Standardize the integration process before AWS deployment.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Created the initial GitHub Actions CI workflow and configured triggers for project branches. Added source checkout and Python/Node.js setup steps.	29/06/2026	29/06/2026	GitHub Actions documentation
3	Added backend dependency installation and automated pytest execution. Reviewed test discovery, module import paths, and the backend working directory used by GitHub Actions.	30/06/2026	30/06/2026	Pytest usage
4	Added frontend dependency installation with `npm ci` and the React production build. Configured `VITE_API_BASE_URL=/api` for the deployed environment.	01/07/2026	01/07/2026	npm ci documentation
5	Added validation for Docker, deployment, and environment configuration files. Improved workflow logs so failures could be traced to a specific validation step.	02/07/2026	02/07/2026	Workflow syntax
6	Tested the workflow with team changes, reviewed failed jobs, corrected YAML and path issues, and documented the integration procedure for future commits.	03/07/2026	03/07/2026	Troubleshooting workflows

Week 3 Achievements

- Created an automated CI workflow for the CloudLibrary repository.
- Automated backend API testing and frontend production building.

- Added configuration validation before deployment.
- Improved the traceability of integration failures through workflow logs.
- Established a repeatable code-validation process for the team.

1.4 Week 4 Worklog

Period: 06/07/2026 – 10/07/2026

Week 4 Objectives

- Prepare AWS identity, permissions, and container repositories.
- Securely connect GitHub Actions to AWS through OpenID Connect.
- Test the Docker image build and push process with Amazon ECR.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Installed and configured AWS CLI, verified the AWS account identity, and standardized the working region as `ap-southeast-2`.	06/07/2026	06/07/2026	AWS CLI installation
3	Created the private ECR repositories `aws-library-backend` and `aws-library-frontend`. Defined image tags and repository naming rules.	07/07/2026	07/07/2026	Amazon ECR documentation
4	Configured the GitHub OpenID Connect provider and created the IAM role `aws-library-github-deploy` with a branch-restricted trust policy.	08/07/2026	08/07/2026	GitHub OIDC with AWS
5	Prepared IAM permissions for ECR, S3, Elastic Beanstalk, CloudFormation, and Auto Scaling. Used policy simulation to identify missing actions.	09/07/2026	09/07/2026	IAM policy simulator
6	Built, tagged, and pushed the frontend and backend Docker images to ECR. Verified image tags, digests, and the ability of AWS resources to pull the images.	10/07/2026	10/07/2026	Pushing images to ECR

Week 4 Achievements

- Configured AWS CLI and a consistent AWS working region.
- Created two private Amazon ECR repositories.
- Connected GitHub Actions to AWS through OIDC without permanent access keys.

- Created the deployment IAM role and tested its permissions.
- Successfully pushed frontend and backend Docker images to ECR.

1.5 Week 5 Worklog

Period: 13/07/2026 – 17/07/2026

Week 5 Objectives

- Create the Elastic Beanstalk application and Docker environment.
- Prepare the deployment bundle and application-version process.
- Complete the first AWS deployment of the integrated project.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Created the Elastic Beanstalk application `aws-library-system` and the Docker environment `Aws-library-system-env`. Reviewed the generated EC2 and environment resources.	13/07/2026	13/07/2026	Elastic Beanstalk documentation
3	Prepared the production Docker Compose deployment template with ECR image URIs, container dependencies, health checks, ports, and environment variables.	14/07/2026	14/07/2026	Beanstalk Docker configuration
4	Prepared the Elastic Beanstalk deployment bundle, uploaded it to the Beanstalk S3 bucket, and created a traceable application version.	15/07/2026	15/07/2026	Application versions
5	Configured Elastic Beanstalk environment properties for PostgreSQL, application secrets, logging, and region settings. Ensured that the backend and database used the same credentials.	16/07/2026	16/07/2026	Environment properties
6	Performed the first deployment, checked the public environment URL and health status, reviewed deployment events, and practised selecting a stable application version for rollback.	17/07/2026	17/07/2026	Deploying an existing version

Week 5 Achievements

- Created the CloudLibrary Elastic Beanstalk application and environment.
- Prepared a Docker Compose deployment package using ECR images.
- Configured application environment variables outside the source code.
- Created and deployed Elastic Beanstalk application versions.
- Understood the deployment-event and rollback mechanisms.

1.6 Week 6 Worklog

Period: 20/07/2026 – 24/07/2026

Week 6 Objectives

- Automate the complete deployment process with GitHub Actions.
- Harden IAM permissions and deployment verification.
- Confirm that a push to `develop_2.0` can update the AWS environment automatically.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Extended the deployment workflow to build the frontend and backend Docker images and push commit-tagged images to the two ECR repositories.	20/07/2026	20/07/2026	Publishing Docker images
3	Automated generation of the deployment bundle, upload to S3, creation of a new Elastic Beanstalk application version, and environment update.	21/07/2026	21/07/2026	Create application version
4	Added a deployment verification loop to compare the expected application version with the running version and check for `Status: Ready` and `Health: Ok`.	22/07/2026	22/07/2026	Describe environments
5	Investigated missing IAM permissions for CloudFormation, S3, and Auto Scaling. Updated the GitHub deployment role and EC2 instance role with the required least-privilege access.	23/07/2026	23/07/2026	Elastic Beanstalk roles
6	Tested the automated pipeline by integrating team changes and pushing them to `develop_2.0`. Verified image publication, application-version creation, deployment execution, and rollback readiness.	24/07/2026	24/07/2026	Monitoring GitHub workflows

Week 6 Achievements

- Completed the main GitHub Actions deployment pipeline.

- Automated Docker image publishing and Elastic Beanstalk deployment.
- Added post-deployment version and health verification.
- Corrected IAM permissions required by the deployment process.
- Confirmed that pushes to `develop_2.0` could trigger AWS deployment automatically.

1.7 Week 7 Worklog

Period: 27/07/2026 – 30/07/2026

Week 7 Objectives

- Configure CloudWatch monitoring, logs, metrics, and alarms.
- Troubleshoot the remaining CI/CD and Elastic Beanstalk deployment problems.
- Complete final integration, deployment validation, and project documentation.

Tasks to be carried out this week

Day	Task	Start Date	Completion Date	Reference Material
2	Configured CloudWatch log collection, metric filters, dashboard widgets, and alarms for CPU utilization, request count, HTTP 5xx responses, and API latency.	27/07/2026	27/07/2026	Amazon CloudWatch documentation
3	Verified application and deployment logs from Elastic Beanstalk, Docker, Nginx, Flask, and Gunicorn. Tested CloudWatch alarms and reviewed the environment health information.	28/07/2026	28/07/2026	Beanstalk logging
4	Resolved CI and deployment issues, including the backend Python import path, missing AWS permissions, application-version mismatch, and PostgreSQL credential inconsistency. Re-ran the workflow after each fix.	29/07/2026	29/07/2026	Elastic Beanstalk troubleshooting
5	Integrated the final frontend and backend changes, updated the book-cover assets, pushed the final version to `develop_2.0`, verified ECR and Elastic Beanstalk deployment, captured evidence, and completed the Workshop and Worklog report sections.	30/07/2026	30/07/2026	Managing Beanstalk environments

Week 7 Achievements

- Completed CloudWatch monitoring, dashboards, log collection, and alarms.

- Diagnosed deployment failures through GitHub Actions, CloudShell, Elastic Beanstalk events, and log bundles.
- Fixed the backend import path, IAM permissions, and PostgreSQL configuration issues.
- Integrated and deployed the final team deliverables.
- Verified the complete flow from source-code push to the running AWS environment.
- Completed the architecture, deployment, troubleshooting, and worklog documentation for the final report.

1.8 Overall Worklog Summary

During the seven-week project period, my responsibilities focused on system integration, containerization, AWS deployment, CI/CD automation, monitoring, and troubleshooting. I integrated the React frontend, Flask backend, and PostgreSQL database using Docker Compose, then configured GitHub Actions to test, build, and deploy the application automatically.

The frontend and backend Docker images were stored in Amazon ECR and deployed to AWS Elastic Beanstalk. GitHub Actions authenticated with AWS through OIDC and IAM roles, while Amazon CloudWatch was used for log collection, health monitoring, and operational metrics.

Several deployment issues, including incorrect import paths, insufficient IAM permissions, and inconsistent database credentials, were identified and resolved through GitHub Actions logs, Elastic Beanstalk events, CloudWatch, and AWS CloudShell.

Overall, my contribution established a repeatable and traceable deployment workflow that connected the frontend and backend components into a functional cloud-based application.

2 Proposal

2.1 Project Overview

CloudLibrary is a cloud-based library management web application developed as part of the AWS Vietnam Internship Program. The project aims to provide a complete digital solution for managing books, users, borrowing activities, returning processes, reservations, notifications, and administrative operations in a library environment.

The system was designed as a full-stack web application with a React frontend, a Flask backend API, and a PostgreSQL database. In addition to implementing the core business features, the project also focuses on cloud deployment, CI/CD automation, containerization, monitoring, and operational management using AWS services.

The main objective of the project is not only to build a functional library system, but also to apply modern DevOps and cloud-native practices. The final system allows the development team to update code, push changes to GitHub, automatically build Docker images, store images in Amazon ECR, deploy the application using AWS Elastic Beanstalk, and monitor the running system through Amazon CloudWatch.

2.2 Project Objectives

The project was planned with the following objectives:

- Build a complete web-based library borrowing and management system.
- Implement authentication and role-based authorization for administrators and normal users.
- Provide book management features such as creating, reading, updating, deleting, restoring, importing, and exporting book data.
- Allow users to borrow books, request returns, renew borrow records, and reserve unavailable books.
- Provide administrators with tools to manage users, books, borrow records, reservations, notifications, and system activities.
- Containerize both frontend and backend applications using Docker.
- Build an automated CI/CD workflow using GitHub Actions.
- Store frontend and backend Docker images in Amazon Elastic Container Registry.
- Deploy the web application on AWS Elastic Beanstalk.
- Monitor application logs, errors, health status, CPU usage, memory usage, and system metrics using Amazon CloudWatch.
- Use AWS CloudShell for quick AWS command-line operations and permission setup.

2.3 Planning

The project was planned as a step-by-step development process. The team first focused on the core application features, then moved to authentication, role-based access control, advanced library operations, DevOps automation, and AWS deployment.

2.3.1 Phase 1: Core Application Development

In the first phase, the team developed the basic library features. The backend was implemented using Flask and Flask-SQLAlchemy, while the frontend was developed using React and Vite. PostgreSQL was selected as the relational database because it is reliable, widely used, and suitable for storing structured data such as users, books, borrow records, reservations, and notifications.

The main features in this phase included:

- Displaying the list of books.
- Viewing book details.
- Creating, updating, and deleting books.
- Managing book availability.
- Creating initial database models.
- Building basic API endpoints for frontend integration.

2.3.2 Phase 2: Authentication and Authorization

After completing the core features, the team implemented authentication and authorization. JSON Web Token was used for user authentication. The backend provides login, registration, current-user information, password reset, and role-based permission checking.

The system includes two main roles:

- **Admin**: has permission to manage books, users, borrow records, reservations, reports, and system operations.
- **User**: can browse books, borrow books, return books, request renewals, reserve books, update profile information, and view personal borrowing history.

This phase was important because it separated administrative operations from normal user operations and improved the security of the system.

2.3.3 Phase 3: Borrowing, Returning, Reservation, and Notification Features

The next phase focused on implementing the main library business workflows. Users can borrow available books, provide customer information, request returns, request renewals, and reserve books that are currently unavailable. Administrators can confirm returns, review return conditions, calculate fines, approve or reject renewal requests, and manage reservation queues.

The system also includes notification features so that users can receive updates about borrow records, return requests, reservation status, and administrative actions.

2.3.4 Phase 4: Containerization and CI/CD

After the application features were completed, the team containerized the frontend and backend. Docker was used to package the applications into isolated and reproducible runtime environments.

The CI/CD workflow was built using GitHub Actions. When developers fix code and push changes to GitHub, GitHub Actions automatically builds Docker images for the frontend and backend. These images are then pushed to Amazon Elastic Container Registry.

This phase improved the development workflow because deployment no longer needed to be performed manually. Instead, code changes could be automatically built and prepared for deployment.

2.3.5 Phase 5: AWS Deployment and Monitoring

In the final phase, the application was deployed to AWS. Amazon ECR was used as the container image registry for both frontend and backend Docker images. AWS Elastic Beanstalk was used to deploy and manage the running web application environment.

Amazon CloudWatch was used to monitor the deployed system. It helps the team observe application logs, detect errors, check system health, and monitor resource usage such as CPU, memory, and other operational metrics. AWS CloudShell was used as a browser-based terminal to run AWS CLI commands quickly, especially for permission setup and operational tasks.

2.4 Technology Stack

2.5 Design and Architecture

The CloudLibrary system follows a cloud-based deployment architecture with automated CI/CD and centralized monitoring. The overall design separates the development workflow, container image storage, deployment environment, database layer, user access flow, and monitoring flow.

2.6 Architecture Analysis

Figure 1 illustrates the complete deployment architecture and workflow of the CloudLibrary project on AWS. The architecture is organized into several main parts: source code management, CI/CD automation, container image registry, application deployment, database storage, user access, monitoring, and cloud operations.

2.6.1 Development and Source Code Management

The workflow starts with the development team. Developers fix bugs, implement new features, and push code changes to the GitHub repository. GitHub is used as the central source code management platform for both frontend and backend code.

Layer	Technology / Service
Frontend	React, Vite, Nginx
Backend	Flask, Flask-SQLAlchemy
Authentication	JWT-based authentication
Database	PostgreSQL
Containerization	Docker
CI/CD	GitHub Actions
Container Registry	Amazon ECR
Application Deployment	AWS Elastic Beanstalk
Monitoring and Logging	Amazon CloudWatch
Cloud Operations	AWS CloudShell

Table 1: Technology stack of the CloudLibrary project

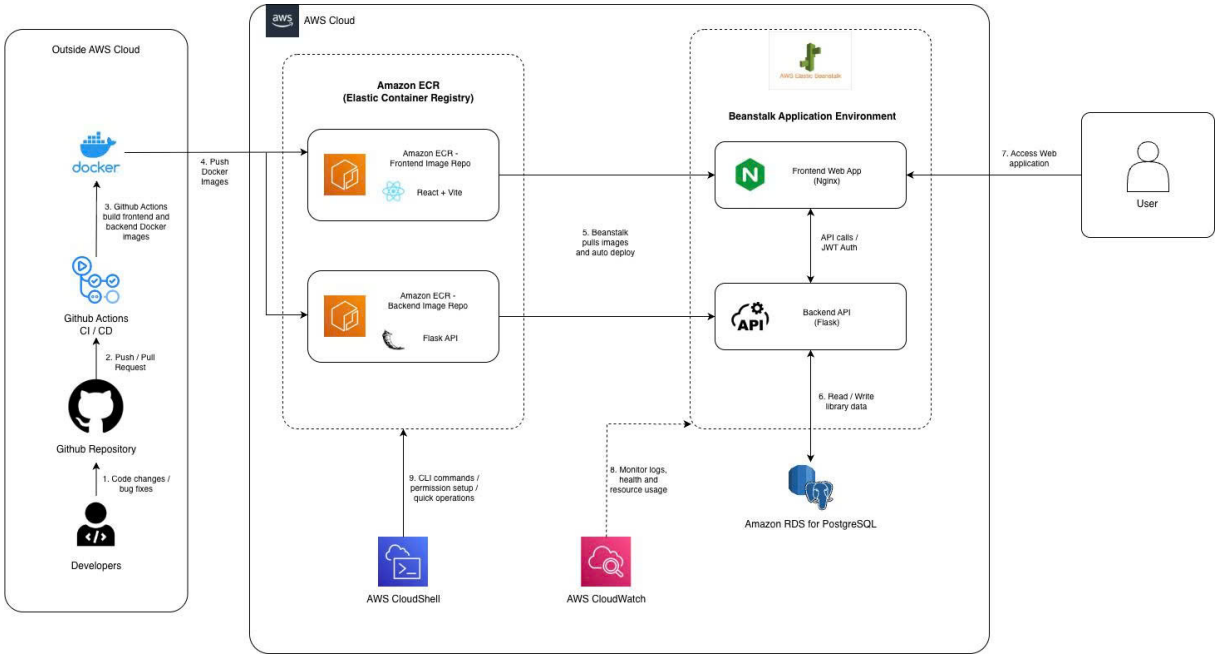

Figure 1: CloudLibrary deployment architecture on AWS

When a developer pushes code or creates a pull request, the CI/CD pipeline is triggered. This ensures that changes are checked and built automatically before being deployed. This approach reduces manual deployment errors and makes the release process more consistent.

2.6.2 CI/CD Pipeline with GitHub Actions

GitHub Actions is responsible for automating the build process. After code is pushed to GitHub, the pipeline builds Docker images for both application layers:

- Frontend Docker image for the React and Vite web application.
- Backend Docker image for the Flask API service.

This process ensures that the frontend and backend are packaged in a consistent runtime environment. By using Docker images, the application can run more reliably across local development, CI/CD, and AWS deployment environments.

2.6.3 Container Image Storage with Amazon ECR

Amazon Elastic Container Registry is used to store Docker images generated by the CI/CD pipeline. The project uses ECR repositories for both frontend and backend images.

The purpose of Amazon ECR in this project is to provide a secure and managed container image registry. After GitHub Actions builds the Docker images, it pushes them to ECR. These images then become the deployment artifacts used by AWS Elastic Beanstalk.

Using ECR helps separate the build stage from the deployment stage. The CI/CD pipeline only needs to build and push images, while the deployment service can later pull the correct images from ECR.

2.6.4 Application Deployment with AWS Elastic Beanstalk

AWS Elastic Beanstalk is used to deploy and manage the CloudLibrary web application. Elastic Beanstalk simplifies deployment by providing a managed application environment. Instead of manually configuring servers, runtime environments, and deployment steps, the team can deploy containerized applications through Beanstalk.

In this architecture, Elastic Beanstalk pulls the frontend and backend Docker images from Amazon ECR and deploys them into the application environment. The frontend web application serves the user interface, while the backend Flask API handles business logic, authentication, book management, borrowing, returning, reservations, notifications, reports, and administrative operations.

Elastic Beanstalk helps the team focus more on application development and less on infrastructure management.

2.6.5 Frontend and Backend Communication

Users access the CloudLibrary system through a web browser. The frontend communicates with the backend through API calls. Authentication is handled using JWT tokens. After users log in, the frontend stores the token and sends it in the request header when calling protected backend APIs.

The backend validates the JWT token and checks the user's role before allowing access to protected resources. Admin users can access management features, while normal users can access borrowing, reservation, profile, and personal history features.

2.6.6 Database Layer with PostgreSQL

The backend API connects to a PostgreSQL database to store application data. The database contains important entities such as users, books, borrow records, reservations, notifications, audit logs, and password reset tokens.

The backend reads and writes data to the database whenever users perform actions such as logging in, borrowing books, returning books, reserving books, updating profiles, or when administrators manage library operations.

PostgreSQL was selected because the system requires structured relational data and consistent relationships between users, books, borrow records, and reservations.

2.6.7 Monitoring with Amazon CloudWatch

Amazon CloudWatch is used to monitor the deployed application. CloudWatch collects logs and system metrics from the running environment. In this project, CloudWatch is used to observe:

- Application logs.
- System errors.
- CPU usage.
- Memory usage.
- Application health.
- Operational metrics.
- Alarms and monitoring dashboards.

CloudWatch helps the team detect problems after deployment. For example, if the backend returns errors, if the application becomes unhealthy, or if CPU and memory usage increase abnormally, CloudWatch provides visibility for troubleshooting.

This monitoring layer is important because deployment alone is not enough for a production-oriented system. The team also needs a way to observe the system after it is running.

2.6.8 Cloud Operations with AWS CloudShell

AWS CloudShell is used as a browser-based terminal for AWS operations. It allows the team to run AWS CLI commands directly in the AWS console without setting up a local CLI environment.

In this project, CloudShell is useful for quick tasks such as permission setup, checking AWS resources, running deployment-related commands, and performing operational checks. It helps reduce the time needed to manually navigate through different AWS service pages.

2.7 Deployment Workflow

The deployment workflow of the project can be summarized as follows:

1. Developers fix code or implement new features.
2. Code is pushed to the GitHub repository.
3. GitHub Actions automatically builds Docker images for the frontend and backend.
4. The generated Docker images are pushed to Amazon ECR.
5. AWS Elastic Beanstalk pulls the images from Amazon ECR.
6. Elastic Beanstalk automatically deploys the web application.
7. Users access the deployed CloudLibrary application through the browser.
8. The backend reads and writes data to PostgreSQL.
9. Amazon CloudWatch continuously monitors logs, errors, health status, and resource usage.
10. AWS CloudShell is used for quick command-line operations and permission setup.

2.8 Security and Access Control Design

The system includes authentication and authorization to protect application resources. JWT is used to verify user identity. After successful login, users receive an access token. This token must be included in protected API requests.

Role-based access control is applied to separate permissions between administrators and normal users. Admin users are allowed to perform sensitive operations such as managing books, users, borrow records, reservations, reports, and system audit logs. Normal users can browse books, borrow books, request returns, request renewals, reserve books, and manage their own profiles.

This design helps reduce unauthorized access and ensures that users can only perform operations suitable for their roles.

2.9 Scalability and Maintainability Considerations

The project was designed with scalability and maintainability in mind. Docker makes the runtime environment more consistent. GitHub Actions improves delivery automation. Amazon ECR provides a centralized container image registry. Elastic Beanstalk simplifies deployment and environment management. CloudWatch provides operational visibility.

Although the current version focuses on deployment using Elastic Beanstalk, the architecture can be extended in the future. Possible improvements include using Amazon RDS as the managed production database, adding Amazon S3 for static assets or uploaded files, integrating Amazon SES for email notifications, and migrating the containerized services to Amazon ECS or Amazon EKS if the system requires more advanced scaling and orchestration.

2.10 Expected Outcomes

The expected outcomes of the proposal are:

- A complete web application for library management.
- A functional authentication and role-based authorization system.
- A complete borrowing, returning, reservation, and notification workflow.
- A Dockerized frontend and backend.
- An automated CI/CD workflow from GitHub to AWS.
- Docker images stored securely in Amazon ECR.
- Application deployment managed by AWS Elastic Beanstalk.
- Monitoring and troubleshooting support through Amazon CloudWatch.
- Faster operational tasks using AWS CloudShell.

2.11 Conclusion

The CloudLibrary proposal focuses on building a practical cloud-based library management system while applying AWS deployment and DevOps practices. The architecture combines application development, containerization, CI/CD automation, AWS deployment, and monitoring into one complete workflow.

Through this design, the project demonstrates how a full-stack web application can be developed locally, containerized with Docker, integrated with GitHub Actions, stored in Amazon ECR, deployed using AWS Elastic Beanstalk, and monitored by Amazon CloudWatch. This provides both a functional application for library operations and a practical learning experience in cloud-native software delivery on AWS. “‘

3 Blogs Posted

Blog 1 - BUILDING A TOKEN MANAGEMENT AND AUTOMATIC SESSION REFRESH MECHANISM

This article introduces the authentication and session handling model in a React Fetch API application, allowing flexible token storage between localStorage or sessionStorage depending on the user's preference. The system integrates a centralized request layer that automatically attaches the Authorization header and handles errors automatically upon encountering an HTTP 401 Unauthorized status code, thereby dispatching a global event for safe redirection without crashing the application.

Blog 2 - OPTIMIZING MULTI-CRITERIA SEARCH AND DATA FILTERING

This article focuses on analyzing techniques to optimize client-side performance for large lists using the useMemo hook combined with intelligent multi-field search string combination (title, author, category, ISBN, location). Additionally, the system applies a combined multi-condition status filtering mechanism and flexible sorting algorithms supporting Vietnamese standards via localeCompare, enhancing a smooth lookup experience for users.

Blog 3 - BUILDING THE BOOK BORROWING - RETURNING BUSINESS WORKFLOW AND CONDITION VALIDATION

This article presents the design model for the digital library business workflow using a multi-step modal form, helping to strictly check conditions before confirming transactions such as return deadlines, deposits, and reader information. At the same time, the flexible role-based UI/UX architecture helps clearly delineate permissions between Administrators (approving renewals, receiving returns, handling fines) and Readers (submitting requests to borrow, return, or request extensions) securely and transparently.

4 Events Participated

Event 1

- **Event Name:** First Cloud AI Journey – Technical Workshop Series (Securing Web Apps, AWS Certification Roadmap, and SLA Monitoring)
- **Date & Time:** 09:00, July 11, 2026
- **Location:** Floor 26, Bitexco Financial Tower, Ho Chi Minh City

- **Role:** Attendee
- **A brief description of the event's content and main activities:**
  - **Content 1 (Securing Web Apps):** Explored the "Frontier Agent" powered by Amazon Bedrock, an autonomous security agent capable of planning and executing security tasks across the full lifecycle—including Design Review, Code Security, and Active Penetration Testing.
  - **Content 2 (AWS Certification Roadmap):** Provided a strategic roadmap for conquering the AWS Certified Cloud Practitioner (CLF-C02) exam, breaking down the four core domains, exam structure, test formats (Pearson VUE or online), and practical tips and tricks.
  - **Content 3 (SLA and Monitoring):** Discussed the significance of Service Level Agreements (SLA) in risk management, the gap between "healthy infrastructure" and a "happy user experience", and how to set up effective monitoring and alerting flows using CloudWatch and SNS.
- **Outcomes or value gained (lessons learned, new skills, contribution to the team/project):**
  - Gained advanced knowledge on leveraging AI and autonomous agents to secure modern web applications and automate vulnerability scanning.
  - Acquired a clear roadmap and strategic preparation methods for obtaining foundational AWS certifications.
  - Understood the critical mindset that healthy server metrics do not guarantee good user experience, and learned how to properly monitor business metrics and SLAs to prevent customer-facing issues.

Event 2

- **Event Name:** Building a Graph Knowledge Base with AWS Neptune for GraphRAG
- **Date & Time:** 09:00, June 6, 2026
- **Location:** Floor 26, Bitexco Financial Tower, Ho Chi Minh City
- **Role:** Attendee
- **A brief description of the event's content and main activities:**
  - **Content 1 (Overview of GraphRAG & Limitations of Traditional Vector RAG):** Explored the limitations of Vector RAG when lacking a global data context, while

examining the solution offered by GraphRAG using Knowledge Graphs to help LLMs understand entity relationships clearly.

  - **Content 2 (AWS Neptune - The Heart of the Graph Knowledge Base):** Learned about the architecture of fully managed graph databases on AWS supporting Property Graphs (Apache TinkerPop Gremlin), RDF/SPARQL, and Neptune ML with built-in GNNs.
  - **Content 3 (Building a Knowledge Pipeline for GraphRAG on AWS):** Covered the end-to-end process from data ingestion and extraction, storing graph structures combined with vector search in Neptune, to retrieval and generation for accurate answers.

- **Outcomes or value gained (lessons learned, new skills, contribution to the team/project):**
  - Gained in-depth knowledge of AWS Neptune graph databases and how to apply them to advanced Generative AI systems.
  - Understood how to implement hybrid search models combining vector search and knowledge graphs to optimize RAG accuracy.
  - Acquired practical skills in building data pipelines integrating LLMs (LangChain/LlamaIndex) with graph databases.

5 Workshop: Deploying CloudLibrary to AWS

5.1 Overview

This workshop describes the complete process used to deploy the **CloudLibrary – Library Borrowing System** to Amazon Web Services. The main purpose is to build an automated deployment pipeline that allows a new application version to be tested, containerized, and deployed whenever source code is pushed to GitHub.

CloudLibrary is implemented as a containerized full-stack web application. Its main technical components are:

- **Frontend:** ReactJS and Nginx.
- **Backend:** Python Flask and Gunicorn.
- **Database:** PostgreSQL.
- **Container platform:** Docker and Docker Compose.
- **Source code repository:** GitHub.

- **CI/CD platform:** GitHub Actions.
- **Container registry:** Amazon Elastic Container Registry.
- **Deployment platform:** AWS Elastic Beanstalk.
- **Monitoring platform:** Amazon CloudWatch.

The deployment process is triggered when a new commit is pushed to the `develop_2.0` branch. GitHub Actions first validates the source code, runs backend tests, and builds the frontend. It then builds Docker images, pushes them to Amazon ECR, creates a new Elastic Beanstalk application version, and deploys the new version to the running environment.

5.2 Prerequisites

Before deploying the project, the following tools and resources are required:

- An AWS account with permission to use IAM, Amazon ECR, Elastic Beanstalk, Amazon S3, EC2, and CloudWatch.
- A GitHub repository containing the CloudLibrary source code.
- Git installed on the development computer.
- Docker Desktop and Docker Compose for local testing.
- Python 3 and pip for running the Flask backend and API tests.
- Node.js and npm for building the React frontend.
- AWS CLI for configuring AWS resources and troubleshooting.
- A GitHub Actions IAM role configured through OpenID Connect.

The project uses the following AWS resources:

- AWS Region: `ap-southeast-2`.
- Backend ECR repository: `aws-library-backend`.
- Frontend ECR repository: `aws-library-frontend`.
- Elastic Beanstalk application: `aws-library-system`.
- Elastic Beanstalk environment: `Aws-library-system-env`.
- GitHub Actions IAM role: `aws-library-github-deploy`.
- Elastic Beanstalk EC2 instance role: `aws-elasticbeanstalk-ec2-role`.

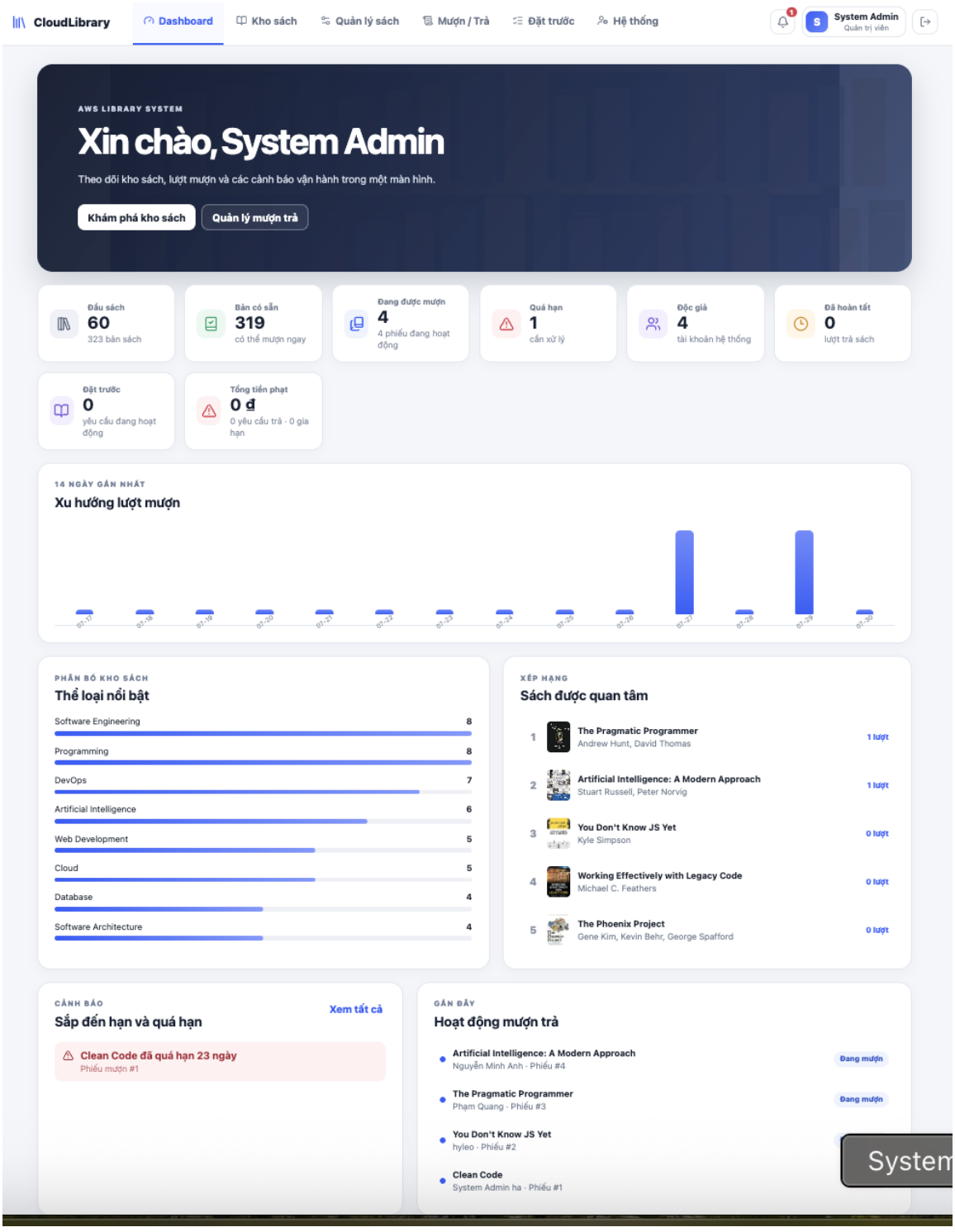

Figure 2: The main interface of the CloudLibrary system

5.3 Deployment Architecture

The deployment architecture of CloudLibrary is illustrated in Figure 3.

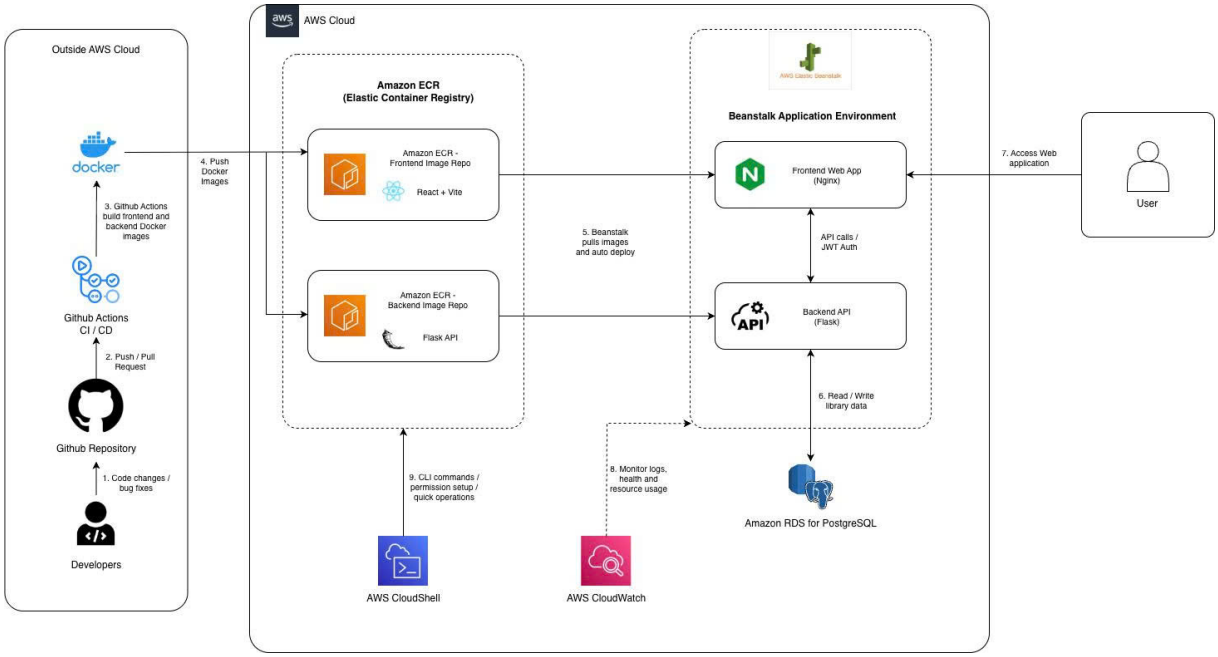

Figure 3: Deployment architecture of CloudLibrary on AWS

The responsibilities of the main services are:

- **GitHub** stores the source code and triggers the deployment pipeline when a new commit is pushed.
- **GitHub Actions** automates testing, Docker image building, image pushing, and Elastic Beanstalk deployment.
- **Amazon ECR** stores private Docker images for the frontend and backend.
- **Elastic Beanstalk** manages the EC2 instance, application versions, Docker deployment, environment health, and public URL.
- **Amazon S3** stores the deployment bundles used by Elastic Beanstalk. It can also be used later to store images uploaded by users or administrators.
- **Amazon CloudWatch** collects application logs, infrastructure metrics, and deployment information.
- **AWS CloudShell** is used to execute AWS CLI commands, configure IAM permissions, and investigate deployment errors.

5.4 Deployment Procedure

5.4.1 Step 1: Verify the Project Structure

Before deployment, the project structure is checked to ensure that all required files are available.

The important directories and files include:

```
.github/workflows/
.platform/hooks/
aws/
backend/
cloudwatch/
deploy/
frontend/
docker-compose.yml
.env.example
```

The `backend` directory contains the Flask API, while the `frontend` directory contains the React application and Nginx configuration.

The `.github/workflows` directory contains the GitHub Actions workflows. The `deploy` directory contains the Docker Compose template used to generate the Elastic Beanstalk deployment bundle.

Sensitive files such as `.env`, passwords, AWS access keys, and session tokens must not be committed to the GitHub repository.

5.4.2 Step 2: Test the Application Locally

The complete application is tested locally with Docker Compose before it is deployed to AWS.

A local environment file is created from the example configuration:

```
cp .env.example .env
```

The application is then built and started:

```
docker compose up --build -d
```

The status of all containers is checked using:

```
docker compose ps
```

The expected services are:

```
db
backend
frontend
```

The PostgreSQL service must become healthy before the backend starts. The backend must also pass its health check before the frontend becomes available.

The backend health endpoint is tested with:

```
curl http://localhost/health
```

A successful response is similar to:

```
{
  "service": "library-flask-api",
  "status": "ok"
}
```

The local website is then accessed at:

```
http://localhost
```

5.4.3 Step 3: Validate the Backend

The backend tests are executed before creating a deployment.

A Python virtual environment is created:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
```

The required packages are installed:

```
python -m pip install --upgrade pip
python -m pip install -r requirements-dev.txt
```

The backend API tests are then executed:

```
PYTHONPATH=. python -m pytest -q
```

These tests verify important backend operations such as authentication, role-based authorization, book management, borrowing, returning, renewal, reservation, and health-check endpoints.

If any required test fails, the deployment workflow is stopped.

5.4.4 Step 4: Build the Frontend

The React application is built in production mode before it is packaged into a Docker image.

The following commands are used:

```
cd frontend
npm ci
VITE_API_BASE_URL=/api npm run build
```

The command creates the optimized frontend files in the `dist` directory.

The value `/api` is used because Nginx forwards frontend API requests to the Flask backend inside the Docker Compose environment.

5.4.5 Step 5: Create Amazon ECR Repositories

Two private Amazon ECR repositories are used:

- `aws-library-backend` stores backend images.
- `aws-library-frontend` stores frontend images.

The repositories can be created using the AWS Console or AWS CLI:

```
aws ecr create-repository \
  --repository-name aws-library-backend \
  --region ap-southeast-2

aws ecr create-repository \
  --repository-name aws-library-frontend \
  --region ap-southeast-2
```

Each image is assigned a version tag based on the GitHub commit. This makes every Docker image traceable to the source code used to build it.

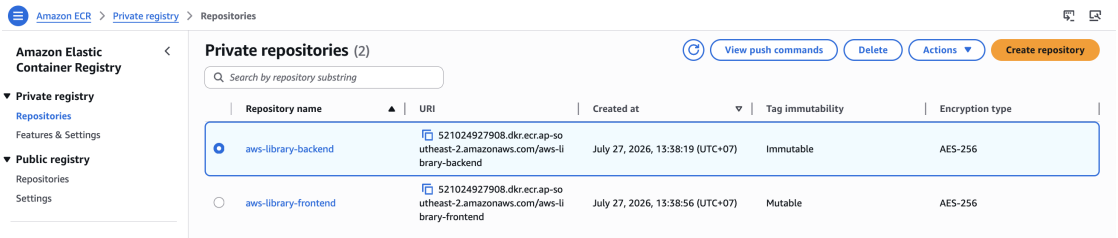

Figure 4: Amazon ECR repositories for the CloudLibrary Docker images

5.4.6 Step 6: Configure GitHub Authentication with AWS

The project uses GitHub OpenID Connect instead of permanent AWS access keys.

GitHub Actions assumes the IAM role:

`aws-library-github-deploy`

The trust policy allows the GitHub repository and the `develop_2.0` branch to request temporary AWS credentials.

The IAM role provides the permissions required to:

- Log in to Amazon ECR.
- Push Docker images.
- Upload the deployment bundle to Amazon S3.
- Create Elastic Beanstalk application versions.
- Update the Elastic Beanstalk environment.
- Read deployment events and environment health information.

Using OpenID Connect avoids storing long-term AWS credentials in GitHub Secrets and improves deployment security.

5.4.7 Step 7: Configure Elastic Beanstalk

An Elastic Beanstalk application named:

`aws-library-system`

is created.

A Docker-based environment named:

`Aws-library-system-env`

is then created using the Docker platform on Amazon Linux.

The environment runs the application with Docker Compose and includes three services:

- `db`: PostgreSQL database.
- `backend`: Flask and Gunicorn API service.
- `frontend`: React application served by Nginx.

The backend and frontend images are pulled from Amazon ECR.

The following environment properties are configured in Elastic Beanstalk:

```
POSTGRES_USER
POSTGRES_PASSWORD
POSTGRES_DB
SECRET_KEY
AWS_REGION
LOG_LEVEL
```

The database URL is generated from the PostgreSQL environment variables:

```
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}
@db:5432/${POSTGRES_DB}
```

This method ensures that the backend and PostgreSQL container use the same credentials.

Passwords and secret values are configured in Elastic Beanstalk and are not stored directly in the GitHub repository.

5.4.8 Step 8: Configure the EC2 Instance Role

The EC2 instance running Elastic Beanstalk uses the role:

`aws-elasticbeanstalk-ec2-role`

The role requires permission to:

- Pull Docker images from Amazon ECR.
- Send logs and metrics to Amazon CloudWatch.
- Communicate with AWS Systems Manager when remote management is used.
- Access Amazon S3 objects required by the deployment.

The following AWS-managed policies can be attached when required:

```
CloudWatchAgentServerPolicy
AmazonSSMManagedInstanceCore
```

If application-uploaded images are stored in Amazon S3, the instance role also requires limited permissions such as `s3:GetObject`, `s3:PutObject`, and `s3:DeleteObject` for the application image bucket.

5.4.9 Step 9: Configure the GitHub Actions Workflow

The deployment workflow is configured to run when new code is pushed to:

`develop_2.0`

The workflow performs the following operations:

1. Checks out the source code.
2. Installs Python dependencies.
3. Runs backend API tests.
4. Installs Node.js dependencies.
5. Builds the React frontend.
6. Requests temporary AWS credentials through OpenID Connect.
7. Logs in to Amazon ECR.
8. Builds the backend Docker image.
9. Builds the frontend Docker image.
10. Pushes both Docker images to Amazon ECR.
11. Generates the Elastic Beanstalk deployment bundle.
12. Uploads the deployment bundle to Amazon S3.
13. Creates a new Elastic Beanstalk application version.
14. Updates the Elastic Beanstalk environment.
15. Waits for the deployment to finish.
16. Verifies the environment health and application version.

Figure 5: GitHub Actions workflow used to validate and deploy CloudLibrary

5.4.10 Step 10: Push the Source Code to GitHub

After the application has passed local testing, the source code is pushed to the deployment branch.

The following commands are executed in the local Git repository:

```
git switch develop_2.0
git add -A
git commit -m "feat: deploy CloudLibrary to AWS"
git push origin develop_2.0
```

The push event automatically starts the GitHub Actions workflow.

No manual Docker image upload or manual Elastic Beanstalk deployment is required after the pipeline has been configured.

5.4.11 Step 11: Build and Push Docker Images

GitHub Actions builds the backend and frontend images separately.

The backend image contains:

- Python runtime.
- Flask source code.
- Gunicorn server.
- Backend dependencies.

The frontend image contains:

- Compiled React files.
- Nginx web server.
- Reverse-proxy configuration for `/api`.
- Static assets and book cover images.

The images are tagged and pushed to the two private ECR repositories.

The workflow uses the commit identifier in the image tag so that the deployed Docker images can be matched with a specific GitHub commit.

5.4.12 Step 12: Create the Deployment Bundle

After the Docker images have been pushed to ECR, GitHub Actions creates an Elastic Beanstalk deployment bundle.

The deployment bundle contains a generated Docker Compose file with the ECR image addresses.

The bundle is compressed into a ZIP file and uploaded to the S3 bucket used by Elastic Beanstalk.

GitHub Actions then creates an application version with a name similar to:

```
gh-<commit-sha>-<workflow-run-id>-<attempt>
```

This naming format makes it possible to identify:

- The GitHub commit used for the deployment.
- The workflow run that created the version.
- The workflow attempt number.

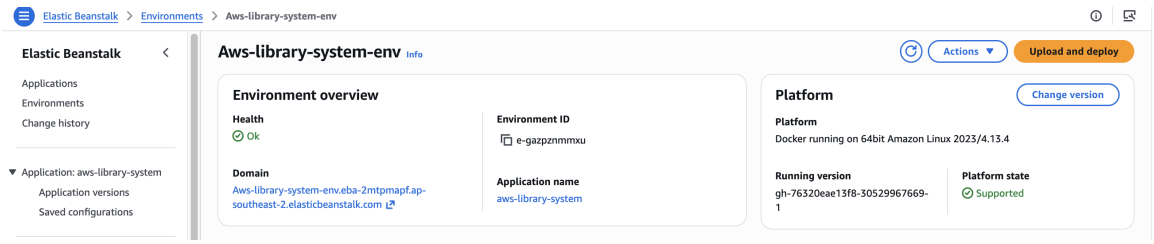

Figure 6: Beanstalk-application-version

5.4.13 Step 13: Deploy the Application

GitHub Actions updates the Elastic Beanstalk environment to use the newly created application version.

During deployment, Elastic Beanstalk performs the following operations:

1. Downloads the deployment bundle from Amazon S3.
2. Reads the Docker Compose configuration.
3. Authenticates with Amazon ECR.
4. Pulls the backend and frontend Docker images.
5. Stops the containers of the previous application version.

6. Starts the PostgreSQL container.
7. Waits for the database health check.
8. Starts the backend container.
9. Waits for the backend health check.
10. Starts the Nginx frontend container.
11. Runs platform deployment hooks.
12. Reports the deployment result to Elastic Beanstalk.

The deployment order is important because the backend depends on PostgreSQL, while the frontend depends on the backend.

5.4.14 Step 14: Verify the Deployment

After Elastic Beanstalk receives the new application version, GitHub Actions checks the environment repeatedly.

The deployment is considered successful when:

- The environment status is `Ready`.
- The environment health is `Ok`.
- The running version matches the newly created version.
- The public health endpoint returns HTTP status code 200.

The expected status is:

```
Status: Ready
Health: Ok
Running version: gh-<new-version>
```

The website can then be accessed through the public Elastic Beanstalk environment URL.

5.4.15 Step 15: Monitor the Application

Amazon CloudWatch is used to collect logs and monitor the running environment.

The monitored information includes:

- EC2 CPU utilization.
- HTTP request count.

- Backend response latency.
- HTTP 4xx and 5xx responses.
- Flask and Gunicorn logs.
- Nginx access and error logs.
- Docker and Docker Compose logs.
- Elastic Beanstalk deployment events.

CloudWatch alarms can be created for conditions such as:

- High CPU utilization.
- High backend latency.
- A large number of HTTP 5xx responses.
- Elastic Beanstalk health changing to `Degraded` or `Severe`.

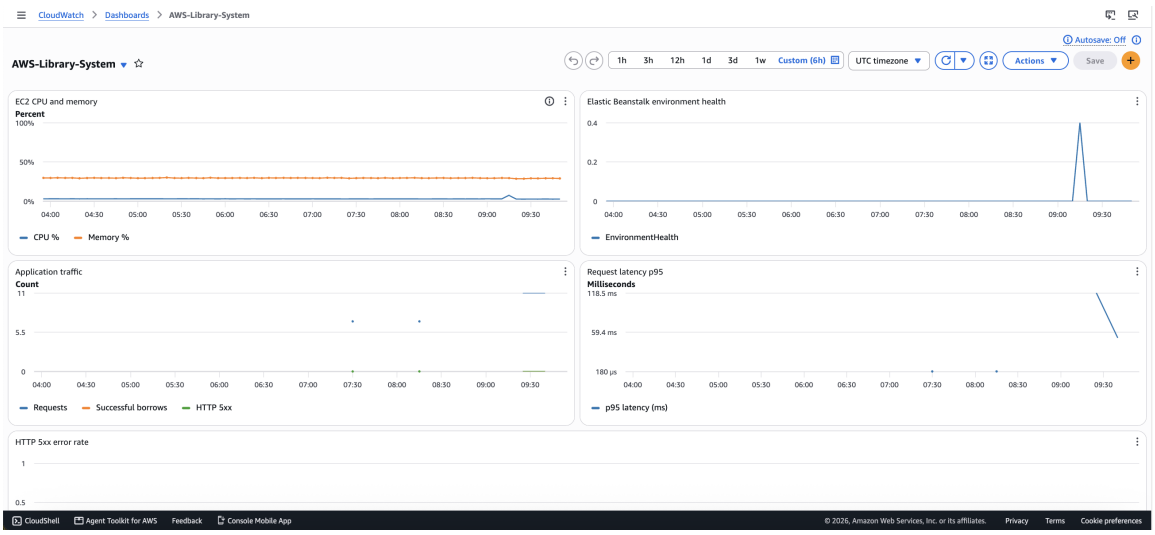

Figure 7: Amazon CloudWatch monitoring for the CloudLibrary environment

5.4.16 Step 16: Troubleshoot Deployment Failures

When a deployment fails, GitHub Actions reports an error in the deployment job. Elastic Beanstalk may also change the environment health to `Degraded`, `Severe`, or `Red`.

The following sources are used to identify the problem:

- GitHub Actions job logs.

- Elastic Beanstalk Events.
- Elastic Beanstalk Deployment Logs.
- Elastic Beanstalk Enhanced Health.
- The `/var/log/eb-engine.log` file.
- Docker container logs.
- CloudWatch Logs.

The Elastic Beanstalk log bundle can be requested with AWS CLI:

```
aws elasticbeanstalk request-environment-info \
  --environment-name Aws-library-system-env \
  --info-type bundle \
  --region ap-southeast-2
```

During the project deployment, one failure occurred because the backend database connection used a hard-coded PostgreSQL password, while the PostgreSQL container used the password configured in Elastic Beanstalk.

The incorrect configuration caused the following sequence:

$$\text{Database credential mismatch} \rightarrow \text{backend connection failure} \rightarrow \text{backend health-check failure} \rightarrow \text{frontend not started} \rightarrow \text{Elastic Beanstalk deployment failure}$$

The problem was resolved by constructing the database URL from:

- `POSTGRES_USER`.
- `POSTGRES_PASSWORD`.
- `POSTGRES_DB`.

After the configuration was corrected, a new commit was pushed to `develop_2.0`, and GitHub Actions started a new deployment.

5.4.17 Step 17: Roll Back to a Stable Version

If a new deployment fails and affects the running environment, a previous stable application version can be deployed again.

The rollback is performed from:

$$\text{Elastic Beanstalk} \rightarrow \text{Application versions} \rightarrow \text{Select a stable version} \rightarrow \text{Deploy}$$

Rolling back allows the website to continue operating while the new version is being corrected.

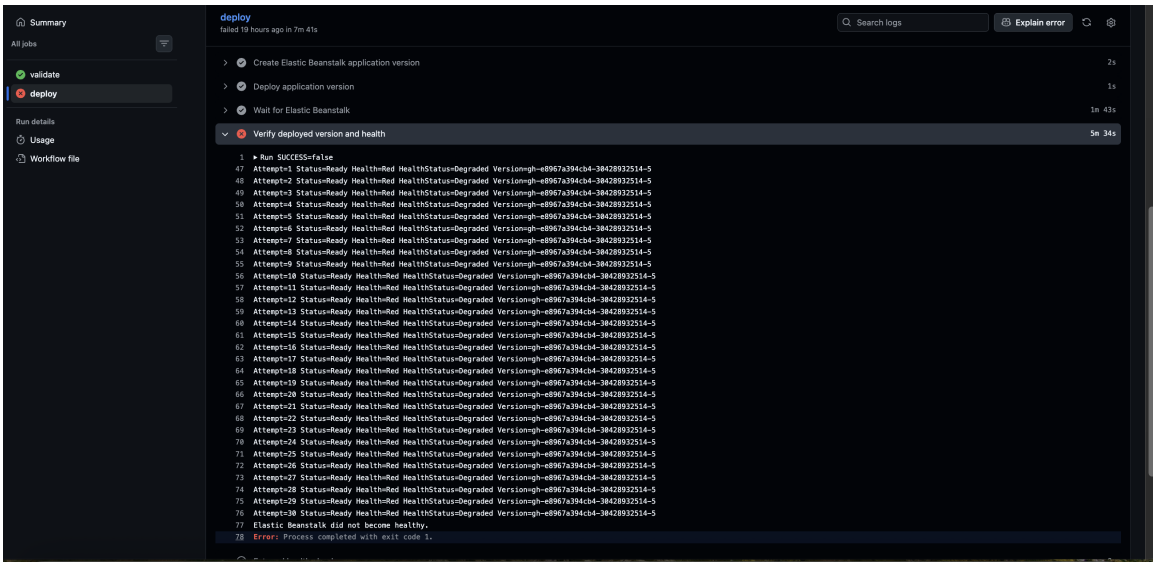

Figure 8: Investigation of a failed Elastic Beanstalk deployment

5.4.18 Step 18: Clean Up Unused Resources

Unused resources should be reviewed regularly to reduce storage usage and AWS cost.

The clean-up process may include:

- Removing old Docker images from Amazon ECR.
- Removing unused Elastic Beanstalk application versions.
- Removing obsolete deployment bundles from Amazon S3.
- Removing unnecessary CloudWatch log groups.
- Removing unused IAM policies and roles.

Local containers can be stopped with:

```
docker compose down
```

The local PostgreSQL volume can also be removed when a clean database is required:

```
docker compose down -v
```

Resources used by the currently running application must not be deleted.

5.5 Deployment Result

After completing the deployment process, the project achieved the following results:

- The frontend and backend were packaged into independent Docker images.

- Docker images were stored in private Amazon ECR repositories.
- GitHub Actions automatically tested and built the application.
- GitHub Actions authenticated with AWS through OpenID Connect.
- Every push to `develop_2.0` triggered the automated deployment process.
- Elastic Beanstalk managed the Docker environment and application versions.
- Amazon CloudWatch collected deployment logs, application logs, and infrastructure metrics.
- Failed deployments could be investigated through GitHub Actions, Elastic Beanstalk logs, CloudWatch, and CloudShell.
- The application could be rolled back to a previous stable version when necessary.

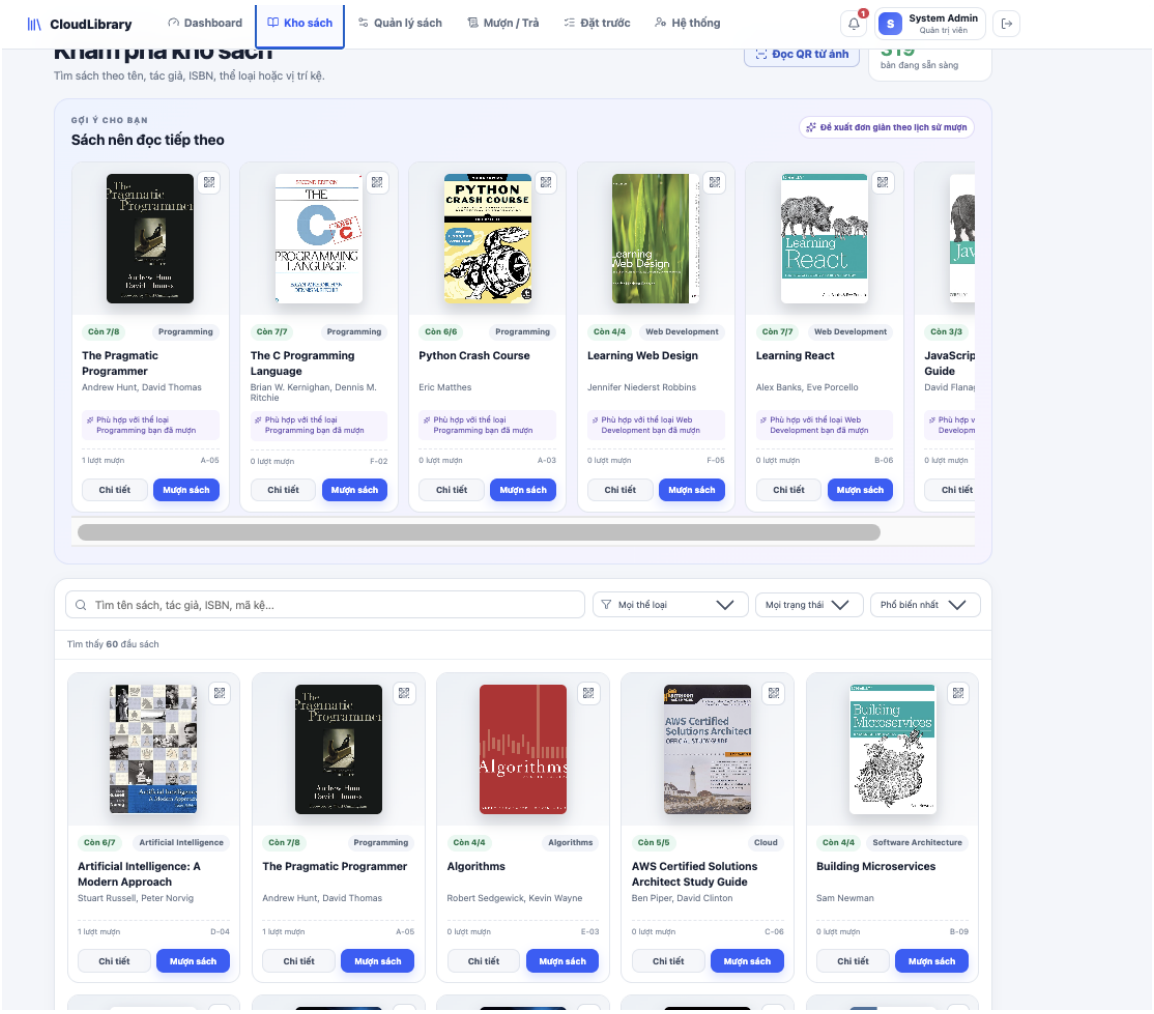

Figure 9: CloudLibrary running on its public Elastic Beanstalk environment

The completed workshop demonstrates a practical deployment pipeline for a containerized full-stack application. The process begins with local validation and continues through GitHub Actions, Amazon ECR, Elastic Beanstalk, and Amazon CloudWatch. As a result, the deployment process becomes repeatable, traceable, and mostly automated.

6 Self-evaluation

During my internship at **Amazon Web Services Viet Nam Company Limited** from **08/06/2026** to **30/07/2026**, I had the opportunity to learn, practice, and apply the knowledge acquired in school to a real-world working environment.

I participated in **developing the CloudLibrary frontend and integrating it with AWS cloud infrastructure**, through which I improved my skills in **React programming, DevOps (Docker, CI/CD), problem-solving, and cross-team communication**.

In terms of work ethic, I always strived to complete tasks well, complied with workplace regulations, and actively engaged with colleagues to improve work efficiency.

To objectively reflect on my internship period, I would like to evaluate myself based on the following criteria:

No.	Criteria	Description	Good	Fair	Avg.
1	**Professional knowledge & skills**	Understanding of the field, applying knowledge in practice, proficiency with tools, work quality	✓		
2	**Ability to learn**	Ability to absorb new knowledge and learn quickly	✓		
3	**Proactiveness**	Taking initiative, seeking out tasks without waiting for instructions	✓		
4	**Sense of responsibility**	Completing tasks on time and ensuring quality	✓		
5	**Discipline**	Adhering to schedules, rules, and work processes	✓		
6	**Progressive mindset**	Willingness to receive feedback and improve oneself	✓		
7	**Communication**	Presenting ideas and reporting work clearly		✓	
8	**Teamwork**	Working effectively with colleagues and participating in teams	✓		
9	**Professional conduct**	Respecting colleagues, partners, and the work environment	✓		
10	**Problem-solving skills**	Identifying problems, proposing solutions, and showing creativity		✓	
11	**Contribution to project/team**	Work effectiveness, innovative ideas, recognition from the team	✓		
12	**Overall**	General evaluation of the entire internship period	✓		

Needs Improvement

- Strengthen discipline and strictly comply with the rules and regulations of the company or any organization.
- Improve problem-solving thinking.

- Enhance communication skills in both daily interactions and professional contexts, including handling situations effectively.

7 Sharing and Feedback

7.1 Overall Evaluation

- **Working Environment:** The environment at AWS Viet Nam is incredibly professional, innovative, and welcoming. The First Cloud AI Journey (FCAJ) members are highly supportive and always willing to share their expertise. The workspace is inspiring and perfectly equipped, allowing me to maintain high focus and productivity throughout the internship.

- **Support from Mentor / Team Admin:** My mentor provided exceptional guidance, especially when I was navigating complex cloud concepts like Docker, ECR, and CI/CD pipelines. They encouraged independent problem-solving while always being available to unblock me. The admin team was equally fantastic, ensuring a seamless onboarding process and providing all necessary resources promptly.

- **Relevance of Work to Academic Major:** Building the frontend for the CloudLibrary project perfectly aligned with my university coursework in software engineering. Furthermore, it introduced me to modern cloud infrastructure and DevOps practices, bridging the gap between academic theory and real-world industry standards.

- **Learning & Skill Development Opportunities:** I acquired a wealth of practical skills during this period. Technically, I leveled up in React, Vite, and containerization. Professionally, I developed crucial soft skills such as agile workflow adaptation, effective communication in a corporate setting, and structured problem-solving.

- **Company Culture & Team Spirit:** The culture here is outstandingly positive and inclusive. There is a strong sense of ownership, mutual respect, and collaboration. Even as an intern, my contributions were recognized and valued, which made me feel like an integral part of the team.

- **Internship Policies / Benefits:** The program provides excellent support for interns, offering flexible working conditions and invaluable access to internal training sessions and AWS resources. These benefits greatly accelerated my learning curve.

7.2 Additional Questions

- **What did you find most satisfying during your internship?**
  Successfully deploying the CloudLibrary frontend to AWS Elastic Beanstalk and seeing

the automated CI/CD pipeline function seamlessly was the most rewarding moment of my internship.

- **What do you think the company should improve for future interns?**
  The FCAJ program is already exceptionally well-structured and comprehensive. I highly appreciate the current format and simply hope the team maintains this high standard and continues providing hands-on projects for future cohorts.

- **If recommending to a friend, would you suggest they intern here? Why or why not?**
  Absolutely. I would highly recommend this program because it offers an unparalleled opportunity to work with cutting-edge cloud technologies, learn directly from industry experts, and contribute to meaningful projects.

7.3 Suggestions & Expectations

- **Suggestions to improve the internship experience:**
  The project-based learning approach (like building CloudLibrary) is incredibly effective. Continuing to assign interns to end-to-end practical projects is the best way to foster growth.

- **Would you like to continue this program in the future?**
  Yes, I would eagerly welcome the opportunity to participate in advanced programs or continue my professional journey with AWS in the future.

- **Any other comments:**
  I want to express my deepest gratitude to my mentor and the entire FCAJ team for their unwavering support and for making this internship a truly transformative experience!